

Liệt kê cây có hướng

Li Yuan

Nguồn gốc: Enumerating directed trees

IOI China candidate team papers / 2001 / Li Yuan.pdf

1 Dẫn nhập

Cây là một cấu trúc phi tuyến rất quan trọng trong thuật toán máy tính. Ngay cả khi tạm gác những ứng dụng rộng rãi của cây sang một bên, việc suy nghĩ và nghiên cứu riêng hình dạng của cây cũng là một quá trình thú vị và có tính thử thách.

Trong tài liệu này, “cây có hướng” được hiểu là cây có gốc; các cạnh có thể xem như hướng từ gốc xuống con. Với bốn đỉnh, chẳng hạn, ta đã có các dạng cơ bản sau nếu viết L là lá và $T(S_1, \dots, S_k)$ là một gốc có các cây con $S_1, \dots, S_k$:

$$T(T(T(L))), \quad T(T(L, L)), \quad T(T(L), L), \quad T(L, L, L).$$

2 Thuật toán liệt kê

Cách làm thông thường là tìm kiếm rồi khử trùng:

sinh trạng thái $\rightarrow$ xét các trạng thái đẳng cấu $\rightarrow$ so với nghiệm đã có $\rightarrow$ thêm vào tập nghiệm.

Phần còn lại trình bày một thuật toán xây dựng có thể sinh tất cả cây có hướng với số đỉnh và độ sâu cho trước mà không sinh lặp.

Hai ý chính của thuật toán là:

- tính không lặp đến từ một định nghĩa thứ tự giữa các cây;
- tính không sót đến từ quy tắc biến đổi từ cây hiện tại sang cây kế tiếp trong thứ tự đó.

3 Định nghĩa thứ tự của cây

Ta đặt ra một quan hệ “lớn hơn” giữa các cây, nhờ đó các cấu trúc cây khác nhau trở thành một dãy có thứ tự. Khi so sánh hai cây A và B , mọi cây con của chúng cũng được sắp từ lớn đến nhỏ theo chính quan hệ này.

Quy tắc so sánh là:

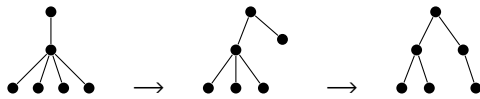
1. Nếu độ sâu của A lớn hơn độ sâu của B , thì $A > B$. Nếu nhỏ hơn, thì $A < B$.
2. Nếu hai độ sâu bằng nhau, so sánh số đỉnh. Cây có nhiều đỉnh hơn là cây lớn hơn.

3. Nếu cả độ sâu lẫn số đỉnh đều bằng nhau, so sánh lần lượt các cây con đã được sắp theo thứ tự giảm dần. Tại vị trí đầu tiên mà hai cây con khác nhau, cây chứa cây con lớn hơn là cây lớn hơn. Nếu cặp cây con hiện tại bằng nhau thì xét cặp kế tiếp.

Như vậy, với mọi d và n , toàn bộ cây có hướng có độ sâu d và số đỉnh n có thể được xếp từ lớn đến nhỏ thành một dãy. Ví dụ với $d = 3, n = 6$, dãy là:

- 1 : $T(T(L, L, L, L))$,
- 2 : $T(T(L, L, L), L)$,
- 3 : $T(T(L, L), T(L))$,
- 4 : $T(T(L, L), L, L)$,
- 5 : $T(T(L), T(L), L)$,
- 6 : $T(T(L), L, L, L)$.

Ba dạng đầu của dãy trên có thể hình dung như sau:



Với d, n bất kỳ, cây đầu và cây cuối của dãy cũng dễ xác định:

- Cây lớn nhất là một đường đi gồm $d - 1$ đỉnh tính từ gốc đến đỉnh ngay trên tầng cuối; đỉnh này có $n - d + 1$ lá.
- Cây nhỏ nhất là một đường đi đạt độ sâu d , còn $n - d$ đỉnh còn lại đều là lá trực tiếp của gốc.

Bài toán vì thế chuyển thành câu hỏi: theo định nghĩa thứ tự trên, có thể tìm một quy tắc biến đổi để bắt đầu từ cây lớn nhất và liên tục sinh cây kế tiếp, không sót và không lặp, cho đến cây nhỏ nhất hay không?

4 Quá trình biến đổi

Chuyển từ một cây trong dãy sang cây kế tiếp là một quá trình làm cây nhỏ đi. Trong quá trình đó, ít nhất một cây con bị làm nhỏ đi. Việc làm nhỏ được thực hiện bằng cách lấy đi một đỉnh con của nó; trong các sơ đồ dưới đây, cây con bị làm nhỏ được đánh dấu bằng khung nét đứt.

Sau khi đỉnh bị lấy đi, một số cây con đứng sau cây con vừa bị làm nhỏ sẽ nhận đỉnh này hoặc được tổ hợp lại. Chúng có thể lớn lên một cách thích hợp. Tuy nhiên, vì trong phép so sánh kích thước cây, vị trí của chúng thấp hơn vị trí của cây con vừa bị làm nhỏ, nên xét toàn cục, cả cây vẫn nhỏ đi.

Quá trình biến đổi được chia thành hai phần:

- Quy trình I: tìm đỉnh sẽ bị xóa.
- Quy trình II: ghép lại đỉnh bị xóa vào các cây con đứng sau.

5 Quy trình I: tìm đỉnh bị xóa

Khi chọn đỉnh bị xóa, ảnh hưởng của việc làm nhỏ cây con chứa nó đối với toàn bộ cây phải nhỏ nhất có thể. Nhờ vậy, sau biến đổi ta thu được đúng cây kế tiếp trong dãy, chứ không nhảy qua nhiều cây.

Từ yêu cầu đó có hai nhận xét trực tiếp:

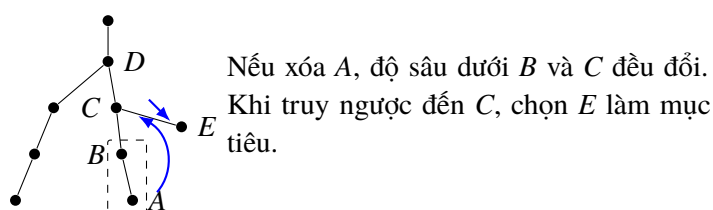
- Đỉnh bị lấy đi chắc chắn là một lá.
- Với nhiều cây con ngang hàng, phải tìm từ phải sang trái.

5.1 Thuật toán của quy trình I

Bắt đầu từ gốc. Trong các con của đỉnh hiện tại, tìm từ phải sang trái con đầu tiên không phải là lá, rồi đi tiếp xuống con đó. Lặp lại cho đến khi gặp một đỉnh mà mọi con của nó đều là lá. Khi đó ứng viên tự nhiên là con cuối cùng của đỉnh này.

Tuy nhiên, chưa thể luôn xem con cuối cùng ấy là đỉnh cần xóa. Giả sử ứng viên A là con duy nhất của cha B . Nếu xóa A khỏi B , độ sâu của cây con gốc B giảm đi 1. Vì trong định nghĩa thứ tự, độ sâu được ưu tiên hơn số đỉnh, ta nên ưu tiên xóa một đỉnh sao cho độ sâu của cây con bị tác động không đổi.

Do đó, sau khi tìm được A , phải truy ngược theo các cha cho đến khi độ sâu của cây con dưới đỉnh hiện tại không đổi khi xóa A . Trong lúc truy ngược, nếu đi qua một đỉnh vẫn còn một con khác, thì bỏ ứng viên cũ và chọn con còn lại đó làm đỉnh bị xóa. Trong sơ đồ dưới đây, khi đi ngược từ A qua B đến C , vì C còn một con E , thuật toán đổi mục tiêu từ A sang E .



6 Quy trình II: ghép lại đỉnh bị xóa

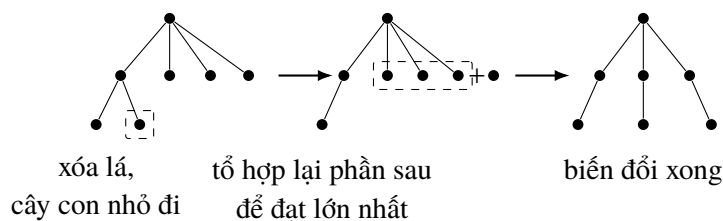
Sau khi tìm và xóa đỉnh mục tiêu, cần xử lý tiếp để tạo thành một cây mới. Nguyên tắc quan trọng là: làm cả cây nhỏ đi, nhưng mức giảm phải là nhỏ nhất.

Sau khi xóa một đỉnh có hai khả năng:

- độ sâu của cây con hiện tại không đổi, còn số đỉnh giảm;
- độ sâu của cây con hiện tại giảm.

6.1 Độ sâu cây con không đổi

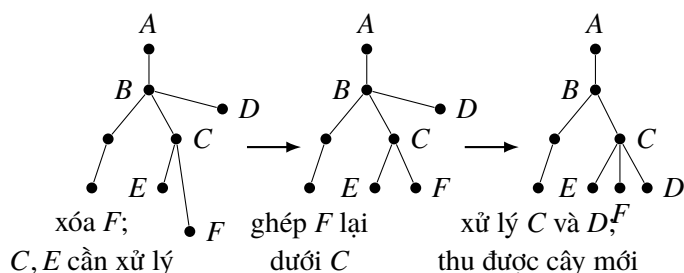
Trong trường hợp này, xét các cây con đứng sau cây con vừa bị sửa. Ta tổ hợp lại chúng, đồng thời đưa đỉnh vừa xóa vào, sao cho mọi cây con ở phần sau không lớn hơn cây con hiện tại và bản thân phần sau là lớn nhất có thể.



6.2 Độ sâu cây con giảm

Nếu xóa đỉnh làm độ sâu của cây con thay đổi, ta phải xử lý theo chuỗi từ dưới lên. Xét ví dụ trong tài liệu: F là đỉnh cần xóa khỏi cha; các cây con gốc C và E sẽ bị giảm độ sâu nên cần xử lý tiếp.

Trước hết xóa F , rồi xử lý E cùng các anh em của nó; ở ví dụ này phần ấy rỗng. Ghép phần này với F và tổ hợp lại dưới gốc C . Tiếp đó xử lý C cùng anh em của nó, chẳng hạn D , và toàn bộ các cây con dưới C ; tổ hợp lại dưới cha B . Sau bước này thu được cây mới.



7 Thuật toán tổng quát

Thuật toán liệt kê cây có gốc có thể viết ở mức cao như sau:

1. Đọc d, n .
2. Tìm cây đầu tiên, tức cây lớn nhất.
3. Trong khi chưa sinh hết các cây:
 4. tìm đỉnh cần xóa $target$;
 5. xóa $target$;
 6. biến đổi cây theo một trong hai trường hợp của Quy trình II.

Điều kiện dừng có thể lấy từ tổng số cây do một thuật toán đếm cung cấp, hoặc có thể xác định trước cây nhỏ nhất rồi dừng khi đạt đến cây đó.

8 Kết luận và độ phức tạp

Mặc dù quá trình biến đổi ở trên trông khá phức tạp, bản chất của nó dựa trên một quy luật gọn và chặt chẽ. Khi nghiên cứu kỹ, có thể thấy nó hài hòa với thứ tự đã định nghĩa, khá giống phép giảm số tự nhiên có mượn.

Chẳng hạn, để biến 2000 thành số tự nhiên nhỏ hơn nó nhưng chênh lệch ít nhất, ta có thể nhìn quá trình như sau:

$$2000 \rightarrow 1000 \rightarrow 1999.$$

Bước đầu tiên làm giảm chữ số quan trọng hơn; bước thứ hai đưa phần phía sau lên lớn nhất có thể. Với cây có hướng cũng vậy: ta tìm một lá từ phải sang trái để xóa, rồi biến đổi các cây con còn lại, tương tự việc đổi các chữ số 0 phía sau thành 9, để mức giảm của toàn cây là nhỏ nhất. Kết quả là cây kế tiếp trong dãy.

Thuật toán trên sinh được, với độ sâu d và số đỉnh n cho trước, mọi hình dạng cây có hướng theo thứ tự từ lớn đến nhỏ. Các thao tác sao chép cây và duyệt cây đều có độ phức tạp $O(n)$. Vì trong quá trình sinh hoàn toàn không có lặp, thời gian chạy chủ yếu phụ thuộc vào tổng số hình dạng cây khác nhau.

Ứng dụng trực tiếp nhất của thuật toán là bài toán cây không gốc. Bảng sau là một tham khảo trực quan về thời gian, so sánh chương trình sinh theo thuật toán xây dựng với phương pháp tìm kiếm. Môi trường thử nghiệm trong tài liệu gốc là PIII 500MHz, 192MB RAM, Borland Pascal 7.0; đơn vị thời gian là giây.

N	11	12	13	14	15	16	17	18	19	20
Xây dựng	0.05	0.05	0.11	0.16	0.49	1.21	3.19	8.52	23.08	62.91
Tìm kiếm	0.27	0.71	2.09	6.04	17.69	51.92	152.20	–	–	–